

Análise crítica e experimental de ferramentas baseadas em Inteligência Artificial na automação de testes: comparação entre EvoSuite (SBST) e LLM na classe *TriangleClassifier*

Ivan Figueiredo Borges

Especialização em Gerência de Projetos de Software na Era de Dados de Sensores e IA

Universidade Federal de Juiz de Fora (UFJF) – Polo Cataguases

Fevereiro de 2026

RESUMO

A Inteligência Artificial aplicada à Engenharia de Software tem ampliado o uso de ferramentas capazes de apoiar atividades do ciclo de vida de desenvolvimento de software, especialmente em codificação, testes e manutenção. No contexto da automação de testes, diferentes abordagens têm sido propostas para reduzir esforço manual, aumentar cobertura e melhorar a qualidade das suítes geradas. Este trabalho apresenta uma análise crítica e experimental de duas abordagens aplicadas à geração de testes unitários para uma classe Java simples, denominada *TriangleClassifier*: a geração automática por *Search-Based Software Testing* (SBST), utilizando EvoSuite, e a geração assistida por um modelo de linguagem de grande porte (LLM), utilizando prompting e refinamento orientado por mutation testing. A avaliação foi realizada com base em métricas de cobertura estrutural, obtidas com JaCoCo, e de cobertura de mutação, obtidas com PIT. Os resultados mostraram que ambas as abordagens alcançaram cobertura estrutural semelhante, com 94% de cobertura de branches. Entretanto, a suíte produzida com auxílio do LLM apresentou melhor desempenho em cobertura de mutação, com 88% de *mutation coverage*, contra 64% obtidos pelo EvoSuite. A análise qualitativa também indicou maior legibilidade, rastreabilidade e manutenibilidade dos testes gerados via LLM. Como conclusão, observa-se que estratégias híbridas, combinando geração assistida por IA com avaliação orientada por métricas fortes, tendem a ser mais eficazes do que o uso isolado de geração automática baseada em busca.

Palavras-chave: Inteligência Artificial na Engenharia de Software; automação de testes; EvoSuite; LLM; mutation testing.

1 INTRODUÇÃO

A crescente incorporação de Inteligência Artificial (IA) em atividades da Engenharia de Software tem transformado a forma como tarefas do ciclo de vida de desenvolvimento de software (SDLC) são executadas. Entre as áreas mais impactadas estão a geração de código, a documentação, a manutenção e, especialmente, a automação de testes. Nesse contexto, ferramentas baseadas em IA vêm sendo utilizadas para reduzir esforço manual, aumentar produtividade e melhorar a qualidade dos artefatos produzidos.

Na etapa de testes, a IA pode atuar tanto por meio de técnicas clássicas de otimização, como o *Search-Based Software Testing* (SBST), quanto por meio de modelos generativos, como os *Large Language Models* (LLMs). Enquanto a primeira abordagem busca automaticamente entradas e sequências de execução que maximizem critérios de cobertura, a segunda utiliza linguagem natural e contexto de código para gerar testes mais próximos da intenção humana.

Este trabalho tem como objetivo comparar, em um estudo de caso controlado, duas abordagens de geração de testes unitários para a classe Java *TriangleClassifier*: (i) geração automática com EvoSuite, representando a linha de SBST, e (ii) geração assistida por LLM, utilizando prompting em duas iterações e refinamento orientado por mutation testing. A análise concentra-se na etapa de teste de software do SDLC, alinhando-se aos eixos de automação de testes e uso de IA generativa/LLMs na Engenharia de Software.

A avaliação foi conduzida com base em duas dimensões complementares: cobertura estrutural, por meio do JaCoCo, e efetividade da suíte em detectar falhas, por meio de mutation testing com PIT. Além da comparação quantitativa, o estudo discute legibilidade, manutenção, desafios técnicos, implicações éticas e tendências futuras para o uso dessas ferramentas no contexto profissional.

2 FUNDAMENTAÇÃO TEÓRICA

A Inteligência Artificial na Engenharia de Software (IAES) pode ser entendida como o uso de técnicas de IA para apoiar ou automatizar tarefas ao longo do ciclo de vida do software, como análise, codificação, testes e manutenção. Na área de testes, uma linha consolidada é o *Search-Based Software Testing* (SBST), em que a geração de testes é formulada como um problema de otimização. Ferramentas como o EvoSuite utilizam algoritmos de busca para produzir suítes que maximizam critérios como cobertura estrutural, buscando cobrir ramos, linhas e caminhos relevantes do código (FRASER; ARCURI, 2011).

Em paralelo, a popularização dos modelos de linguagem de grande porte (*Large Language Models* – LLMs), baseados em arquiteturas do tipo *Transformer* (VASWANI et al., 2017), introduziu uma nova forma de apoio ao desenvolvimento de software. Esses modelos conseguem gerar testes, sugerir refatorações e produzir documentação a partir de instruções em linguagem

natural e contexto de código. Diferentemente do SBST, que opera sobre objetivos formais de busca, os LLMs exploram padrões estatísticos aprendidos em grandes volumes de texto e código, o que favorece testes mais legíveis e semanticamente alinhados às intenções descritas no prompt.

Para avaliar a qualidade das suítes produzidas, este trabalho utilizou duas métricas complementares. A primeira foi a cobertura estrutural, medida com JaCoCo, que indica quais partes do código foram executadas pelos testes. Embora útil, essa métrica não é suficiente para garantir a qualidade semântica de uma suíte, uma vez que executar uma linha ou ramo não significa, necessariamente, verificar adequadamente o comportamento do software. Por esse motivo, a segunda métrica adotada foi o *mutation testing*, com a ferramenta PIT, que introduz pequenas alterações no código e avalia se os testes conseguem detectar essas mudanças. Conforme apontado por Jia e Harman (2011), a cobertura de mutação é uma medida mais forte da efetividade dos testes, pois se aproxima mais da capacidade real de identificar defeitos.

Assim, a combinação entre cobertura estrutural e mutation testing oferece uma base mais robusta para comparar ferramentas baseadas em IA aplicadas à geração de testes.

3 ESTUDO DE CASO E ANÁLISE PRÁTICA

3.1 CENÁRIO EXPERIMENTAL

O estudo de caso foi realizado sobre a classe Java *TriangleClassifier*, responsável por classificar triângulos nos tipos INVALID, EQUILATERAL, ISOSCELES, SCALENE e RIGHT. A lógica implementada inclui validação de lados não positivos, verificação da desigualdade do triângulo, detecção de casos especiais e uso de long para evitar overflow no cálculo dos quadrados dos lados.

Essa classe foi escolhida por apresentar características adequadas para comparação entre abordagens de geração de testes, tais como múltiplos ramos condicionais, casos de borda, condições compostas e lógica aritmética suscetível a mutação.

3.2 FERRAMENTAS UTILIZADAS

O experimento foi executado em um projeto Maven com as seguintes ferramentas:

- Java 11
- JUnit 4.13.2
- JaCoCo 0.8.12
- PIT 1.15.2
- EvoSuite 1.2.0
- LLM: ChatGPT 5.2 Thinking

3.3 FLUXO EXPERIMENTAL

No cenário com EvoSuite, a suíte de testes foi gerada automaticamente e executada sobre a classe alvo. Em seguida, foram produzidos os relatórios de cobertura com JaCoCo e os relatórios de mutação com PIT.

No cenário com LLM, foi utilizado inicialmente um prompt solicitando testes unitários JUnit 4 com foco em cobertura de branches, casos de borda, desigualdade do triângulo, casos típicos e casos retângulos. Após a primeira execução, os relatórios do PIT identificaram mutantes sobreviventes. Em uma segunda iteração, esses mutantes foram fornecidos ao LLM em um novo prompt, solicitando ajustes ou novos testes capazes de aumentar a cobertura de mutação sem gerar redundância excessiva.

3.4 OBSERVAÇÕES DE SETUP E TROUBLESHOOTING

Durante a montagem do experimento, houve um problema inicial em que o JaCoCo reportava 0% de cobertura para a suíte gerada pelo EvoSuite. Esse comportamento foi resolvido com ajustes na configuração do Maven, especialmente na integração entre JaCoCo, Surefire e a execução da suíte *_ESTest. Também foram necessários ajustes relacionados ao runtime do EvoSuite em ambiente moderno de Java, bem como à preservação do argLine injetado pelo JaCoCo. Embora essas dificuldades não tenham comprometido os resultados finais, elas evidenciam que parte do esforço em SBST está na integração ferramental, e não apenas na geração dos testes em si.

4 RESULTADOS E DISCUSSÃO

4.1 RESULTADOS QUANTITATIVOS

A Tabela 1 apresenta a comparação entre as duas abordagens.

Tabela 1 – Comparação entre EvoSuite e LLM

Métrica	EvoSuite (SBST)	LLM (com refinamento por mutação)
Classe de teste principal	TriangleClassifier_EST est	TriangleClassifierTest
Nº de testes executados (Surefire)	10	15
Tempo dos testes (Surefire)	~1,64 s	~0,17 s
JaCoCo – Instruções	98% (122/124)	97% (121/124)
JaCoCo – Branches	94% (17/18)	94% (17/18)
JaCoCo – Linhas	23/24 (~96%)	23/24 (~96%)

JaCoCo – Métodos	100% (3/3)	67% (2/3)
PIT – Line Coverage (classes mutadas)	94% (17/18)	94% (17/18)
PIT – Mutation Coverage	64% (16/25)	88% (22/25)
PIT – Test Strength	67%	88%
PIT – Mutations sem cobertura	1	0
PIT – Testes executados na análise de mutação	128	83

Os resultados mostram que ambas as abordagens alcançaram **cobertura estrutural muito próxima**, especialmente em branches, com 94% em ambos os casos. O EvoSuite apresentou vantagem marginal em cobertura de instruções, com 98% contra 97% do LLM. Entretanto, essa diferença não se traduziu em maior efetividade da suíte.

4.2 DISCUSSÃO DOS RESULTADOS

A principal diferença apareceu no mutation testing. A suíte produzida e refinada com LLM atingiu **88% de mutation coverage**, enquanto a suíte do EvoSuite ficou em **64%**. Em termos absolutos, o LLM matou **22 mutantes**, contra **16** do EvoSuite, um ganho de 6 mutantes mortos. Além disso, o cenário com LLM não apresentou mutações sem cobertura, ao passo que o EvoSuite apresentou uma.

Esse resultado é importante porque indica que **cobertura estrutural não é suficiente para avaliar a qualidade dos testes**. Embora o EvoSuite tenha produzido uma suíte com cobertura estrutural ligeiramente superior, a abordagem baseada em LLM foi mais eficaz em detectar variações defeituosas do programa.

Outro ponto relevante está na quantidade de testes executados pelo PIT: 128 no cenário EvoSuite e 83 no cenário LLM. Isso sugere que, apesar de a suíte assistida por LLM possuir mais métodos de teste, ela foi mais direcionada e menos redundante para os objetivos observados.

4.3 ANÁLISE QUALITATIVA DAS ABORDAGENS

Do ponto de vista qualitativo, a suíte gerada com LLM apresentou vantagens importantes. Os testes foram nomeados de forma descritiva, como `classify_shouldReturnInvalid_whenTriangleInequalityIsViolated_equalCase`, o que facilita leitura, manutenção e rastreabilidade. Também foi possível controlar

melhor a intenção semântica dos testes por meio do prompt, focando explicitamente em casos de borda, regras de negócio e cenários específicos.

Já o EvoSuite mostrou-se eficiente como baseline automatizado, produzindo rapidamente uma suíte com alta cobertura. No entanto, os testes gerados tendem a ser menos expressivos para leitura humana e mais dependentes da infraestrutura de geração e execução. Em um contexto profissional, isso pode aumentar o esforço de manutenção e revisão.

4.4 MUTANTES SOBREVIVENTES NO CENÁRIO LLM

No cenário com LLM permaneceram 3 mutantes sobreviventes, todos associados ao ConditionalsBoundaryMutator na linha 17, correspondente à validação $a \leq 0 \parallel b \leq 0 \parallel c \leq 0$. Esses mutantes alteram a fronteira da condição, como por exemplo substituir ≤ 0 por < 0 .

Mesmo após a adição de testes cobrindo valores zero e negativos em diferentes posições (a, b e c), os mutantes continuaram sobrevivendo. Há fortes indícios de que esses mutantes sejam equivalentes, ou quase equivalentes do ponto de vista comportamental, pois, mesmo quando a primeira validação é enfraquecida, a lógica subsequente de desigualdade do triângulo tende a classificar o caso como INVALID. Assim, a permanência desses três sobreviventes não necessariamente representa falha real da suíte, mas sim uma limitação clássica do mutation testing em lidar com mutantes equivalentes.

4.5 PAPEL DOS PROMPTS NO REFINAMENTO DA SUÍTE LLM

O primeiro prompt solicitou testes JUnit 4 com foco em branch coverage, casos de borda, desigualdade do triângulo, casos típicos e casos retângulos. O resultado foi uma suíte inicial funcional, legível e com boa cobertura estrutural, mas ainda com mutantes sobreviventes no PIT.

O segundo prompt foi elaborado a partir do mutations.xml, instruindo o LLM a gerar novos testes ou ajustar os já existentes para matar os mutantes sobreviventes, mantendo legibilidade e evitando redundância. Esse refinamento foi decisivo para elevar a cobertura de mutação e eliminar mutantes associados a MathMutator, NullReturnVals, NegateConditionals e VoidMethodCallMutator.

Esse resultado sugere que o uso de LLM em fluxo iterativo, orientado por métricas fortes de qualidade, é mais promissor do que o uso pontual em uma única geração.

5 DESAFIOS E ÉTICA

Apesar dos ganhos observados, o uso de ferramentas baseadas em IA na automação de testes apresenta desafios técnicos e éticos relevantes.

No caso dos LLMs, um dos principais riscos é a geração de testes aparentemente corretos, mas semanticamente fracos, redundantes ou

incompletos. Isso pode gerar falsa sensação de qualidade se os resultados forem aceitos sem validação humana. Em ambientes corporativos, também surgem preocupações relacionadas à privacidade de dados, pois trechos de código, mensagens de erro, relatórios de falha e regras de negócio podem ser enviados a serviços externos. Além disso, existem questões de propriedade intelectual, rastreabilidade e governança, especialmente quando a organização precisa justificar a origem de artefatos gerados por IA.

Do ponto de vista de segurança, testes gerados automaticamente podem deixar de contemplar cenários críticos, o que reduz a capacidade de detectar vulnerabilidades ou falhas comportamentais relevantes. Assim, confiar exclusivamente na geração automática, sem validação por métricas e revisão especializada, pode introduzir riscos de qualidade.

No caso do EvoSuite, embora a ferramenta produza rapidamente uma suíte inicial, o experimento mostrou dependência de configuração específica, ajustes de integração e menor legibilidade dos testes gerados. Isso não representa um problema ético no mesmo sentido dos LLMs, mas constitui um desafio prático de adoção e manutenção.

Portanto, o uso ético e eficaz dessas ferramentas requer supervisão humana, validação crítica, métricas objetivas e políticas claras de uso, principalmente quando houver exposição de código sensível ou dependência de plataformas externas.

6 CONCLUSÃO

Este trabalho apresentou uma comparação crítica e experimental entre duas abordagens de geração de testes unitários para a classe *TriangleClassifier*: geração automática por SBST, com EvoSuite, e geração assistida por LLM com refinamento orientado por mutation testing.

Os resultados mostraram que ambas as abordagens alcançaram cobertura estrutural semelhante, especialmente em branches, com 94% em ambos os cenários. No entanto, a suíte produzida com auxílio do LLM apresentou melhor efetividade prática, com 88% de *mutation coverage* e *test strength*, superando os 64% e 67% obtidos pelo EvoSuite. Além disso, os testes gerados via LLM se mostraram mais legíveis, mais fáceis de manter e mais alinhados à intenção semântica do domínio.

A análise dos mutantes sobreviventes também foi relevante. No cenário LLM, permaneceram apenas três mutantes de fronteira associados à validação de lados não positivos, havendo fortes indícios de que sejam mutantes equivalentes. Isso reforça a ideia de que a limitação observada não está necessariamente na qualidade da suíte, mas na própria natureza do mutation testing.

Dessa forma, conclui-se que a estratégia mais promissora não é a substituição do engenheiro de software por ferramentas de IA, mas sim a adoção de fluxos

híbridos. Ferramentas como EvoSuite podem acelerar a geração inicial de uma suíte, enquanto LLMs podem produzir testes mais expressivos e semanticamente orientados, principalmente quando refinados com apoio de métricas mais fortes, como mutation testing.

Em perspectiva futura, o papel do engenheiro de software tende a se transformar. Em vez de apenas escrever testes manualmente, esse profissional passa a atuar também como formulador de prompts, curador de artefatos gerados, analista de métricas, validador de qualidade e responsável por decisões críticas sobre quando confiar — ou não — no apoio automatizado. Nesse sentido, a IA não elimina a expertise humana, mas exige ainda mais julgamento técnico, responsabilidade e capacidade de integração entre ferramentas, métricas e objetivos de negócio.

REFERÊNCIAS

FRASER, Gordon; ARCURI, Andrea. EvoSuite: automatic test suite generation for object-oriented software. In: **PROCEEDINGS OF THE 19TH ACM SIGSOFT SYMPOSIUM AND THE 13TH EUROPEAN CONFERENCE ON FOUNDATIONS OF SOFTWARE ENGINEERING**. New York: ACM, 2011. p. 416-419.

JACOCO. JaCoCo Java Code Coverage Library. [S. l.], [s. d.]. Disponível em: <https://www.jacoco.org/jacoco/>. Acesso em: 26 fev. 2026.

JIA, Yue; HARMAN, Mark. An analysis and survey of the development of mutation testing. **IEEE Transactions on Software Engineering**, [S. l.], v. 37, n. 5, p. 649-678, 2011.

MYERS, Glenford J.; SANDLER, Corey; BADGETT, Tom. **The art of software testing**. 3. ed. Hoboken: John Wiley & Sons, 2011.

PITEST. PIT Mutation Testing. [S. l.], [s. d.]. Disponível em: <https://pitest.org/>. Acesso em: 26 fev. 2026.

VASWANI, Ashish et al. Attention is all you need. In: **ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS**. [S. l.]: Curran Associates, 2017. p. 5998-6008.

ANEXOS

- Repositório do projeto:
<https://github.com/ivanfborges/sbst-vs-llm>
- Pasta com evidências do experimento (relatórios, logs, prompts, prints e versões das suítes):
<https://github.com/ivanfborges/sbst-vs-llm/tree/main/evidencias>